Izveštaj o testiranju performansi sistema (Load Testing)

Projekat: Smart Manufacturing

Kandidat: Marina Ivanović (Student 2)

Alat za testiranje: Locust

1. Uvod i metodologija testiranja

U okviru verifikacije nefunkcionalnih zahteva sistema, sprovedeno je testiranje performansi (Load i Stress testing) korišćenjem alata Locust. Cilj testiranja je ispitivanje ponašanja aplikacije pri različitim nivoima opterećenja, u rasponu od 100 do 10.000 (ili manje, u zavisnosti od testa) istovremenih korisnika.

Za testiranje je odabrano 10 najkritičnijih endpoint-ova, fokusirajući se na one koji su vremenski najzahtevniji (kompleksni upiti, rad sa fajlovima, analitika) i one koji trpe najveći broj konkurentnih zahteva. Metodologija obuhvata postepeno povećanje broja korisnika kako bi se identifikovala uska grla (bottlenecks) i tačka pucanja (breaking point) sistema.

Specifikacija testnog okruženja:

- **Računar:** Lenovo Legion 5 15ARH7H (AMD Ryzen 7 6800H 8GB RAM, NVIDIA GeForce RTX 3060 GPU)
- **OS:** Microsoft Windows 11 Pro
- **Napomena:** Testiranje je vršeno u lokalnom okruženju gde su baza podataka, backend aplikacija i alat za generisanje opterećenja pokrenuti na istoj mašini.

2. Analiza performansi izabranih endpoint-ova

2.1. Login: *POST /auth/login*

Opis i cilj testa:

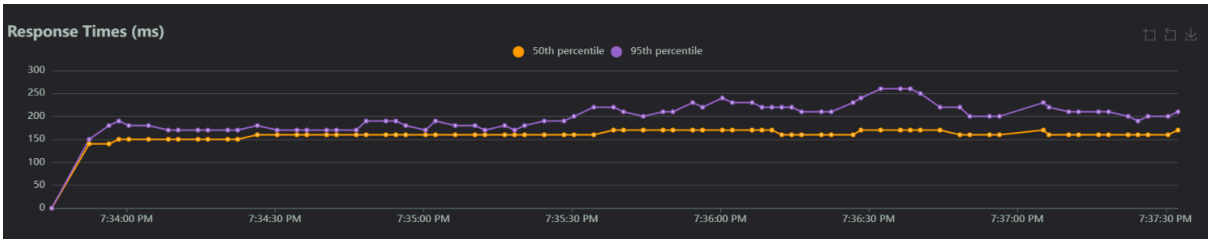
Cilj testiranja ovog endpoint-a je provera stabilnosti i brzine autentifikacionog procesa pod velikim opterećenjem. S obzirom na to da je baza podataka inicijalizovana sa preko 3.000.000 korisničkih zapisa, ovaj endpoint je kritičan za rano otkrivanje uskih grla u vezi sa pretragom baze i kriptografskim operacijama heširanja.

Početne performanse (bez optimizacija):

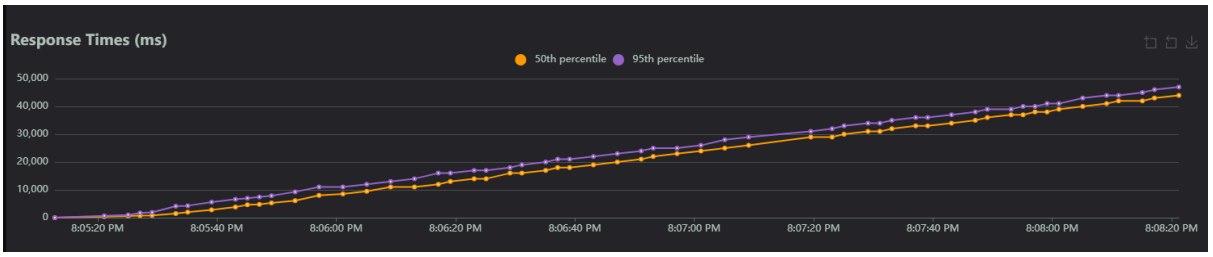
Prilikom inicijalnog testiranja primećena je eksponencijalna degradacija performansi sa povećanjem broja korisnika. Iako za 100 korisnika sistem daje odličan odziv (medijana 160ms), na opterećenju od 5.000 korisnika vreme čekanja (medijana) skače na 40.000ms (40 sekundi), što je daleko van granica prihvatljivog korisničkog iskustva. Nije bilo padova (fails), ali sistem je bio blokiran usled I/O i CPU zagušenja.

Vreme odziva pre optimizacije

- 100 korisnika: Median 160ms, 95%ile 210ms



- 1.000 korisnika: Median 11.000ms, 95%ile 15.000ms
- 2.000 korisnika: Median 15.000ms, 95%ile 29.000ms
- 3.000 korisnika: Median 23.000ms, 95%ile 43.000ms



- 5.000 korisnika: Median 40.000ms, 95%ile 77.000ms

Analiza problema i primenjena rešenja:

Analizom uskih grla ("bottlenecks"), ustanovljena su dva ključna problema:

1. Nedostatak indeksa na tabeli: PostgreSQL je vršio sekvencijalno skeniranje (Sequential Scan) nad tabelom od 3 miliona redova prilikom svake provere imejla.
2. CPU zagušenje zbog Bcrypt algoritma: Najveći uzrok naglog skoka kašnjenja sa povećanjem konkurentnosti bilo je iscrpljivanje CPU resursa ("CPU exhaustion") prilikom BCrypt heširanja.

Rezultati nakon optimizacije:
Nakon implementacije B-Tree indeksa i smanjenja troškova kriptografskih operacija, performanse su značajno unapređene. Kao što se vidi na testu od 10.000 istovremenih korisnika, iako je došlo do iscrpljivanja "connection pool-a" (što je rezultovalo odbijanjem dela konekcija - ConnectionRefusedError), sistem je nastavio da obrađuje validne zahteve sa vremenom značajno kraćim nego ranije.

Vreme odziva nakon optimizacije

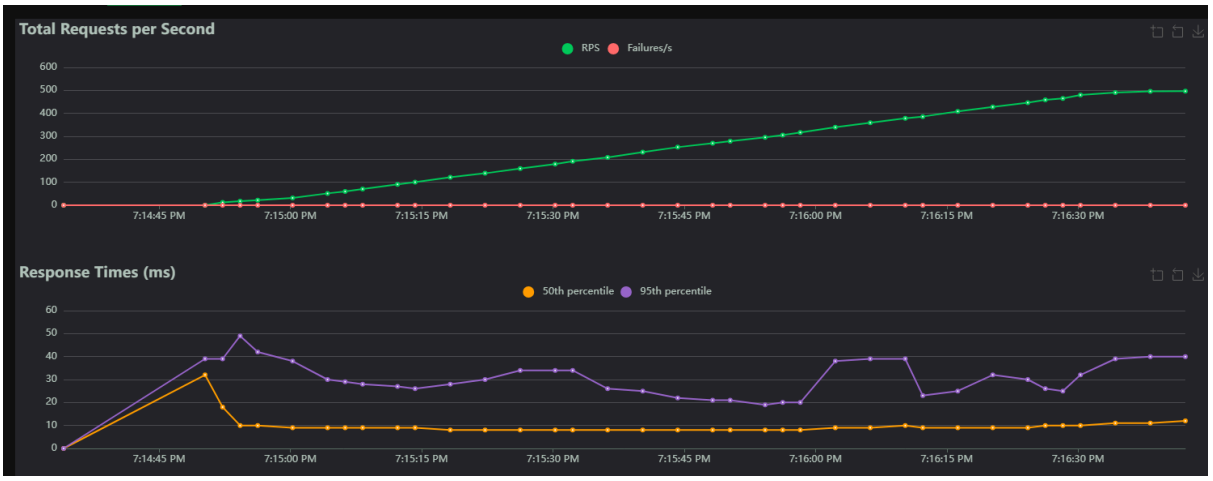
- 10.000 korisnika: Median 2.000ms, 95%ile 4.000ms (Napomena: Ova količina korisnika kada se login kombinuje sa drugim endpoint pozivima, npr. pravljenje rezervacije, uvek prouzrokuje ConnectionRefusedError, samim tim nisam stavila te primere u izveštaj)

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	1. POST /login	273410	0	2000	4300	5500	3395.7	4	381645	445.13	538	0
	Aggregated	273410	0	2000	4300	5500	3395.7	4	381645	445.13	538	0

Druge količine korisnika i performanse login endpointa:

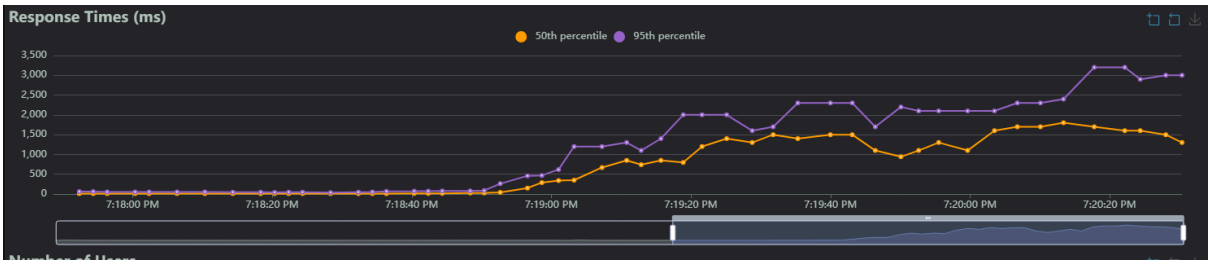
- login za 100 kad sam izmenila tip enkripcije

- login 1000 korisnika:



- login 5000 korisnika

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	1. POST /login	79218	0	490	2100	3200	818.57	5	17862	445.13	555.9	0
	Aggregated	79218	0	490	2100	3200	818.57	5	17862	445.13	555.9	0



- login 8000 korisnika:

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	1. POST /login	256856	0	2100	4300	5100	2552.4	5	338581	445.12	561.1	0
	Aggregated	256856	0	2100	4300	5100	2552.4	5	338581	445.12	561.1	0

2.2. Registracija kupca: *POST /auth/register*

Opis i cilj testa:

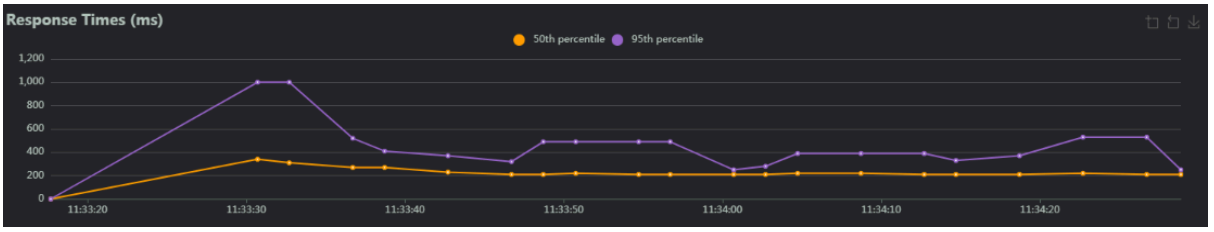
Cilj testiranja ovog endpoint-a je provera sposobnosti sistema da istovremeno kreira veliki broj novih naloga. Proces registracije je po prirodi veoma složen ("heavy") jer zahteva generisanje BCrypt heša za novu lozinku, interakciju sa mrežnim fajl sistemom usled *upload*-a profilne slike, kao i komunikaciju sa spoljnim servisom putem koga se šalje konfirmacioni mejl.

Početne performanse (bez optimizacija):

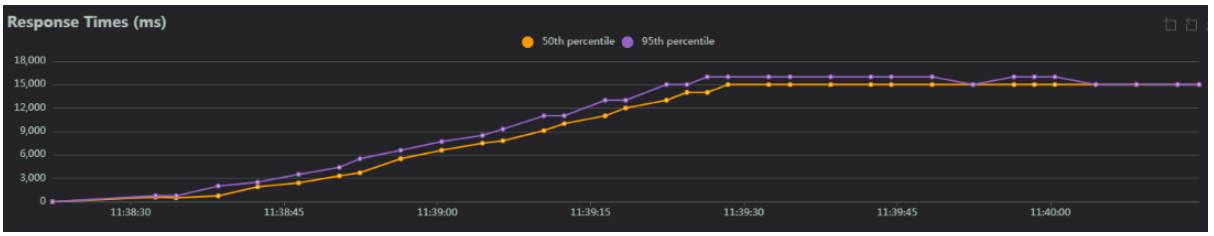
Iako je odziv na 100 korisnika bio prihvatljiv (medijana 210ms), sa povećanjem opterećenja uočena je izuzetno velika latencija. Na opterećenju od svega 2.000 korisnika, medijana odziva je iznosila 21.000ms. Testiranje na višem broju korisnika dovelo je do potpunog kolapsa performansi.

Vreme odziva pre optimizacije

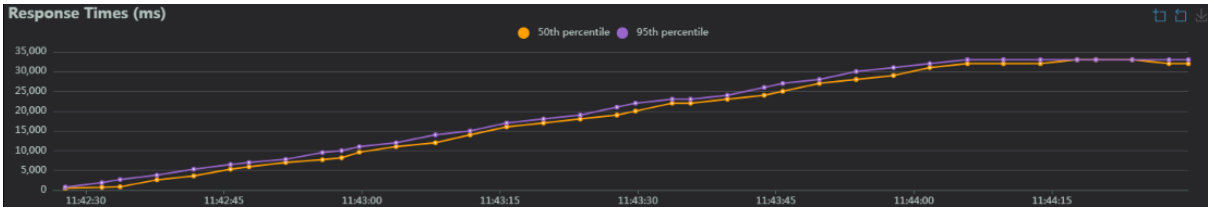
- 100 korisnika: Median 210ms, 95%ile 390ms



- 1.000 korisnika: Median 15.000ms, 95%ile 16.000ms



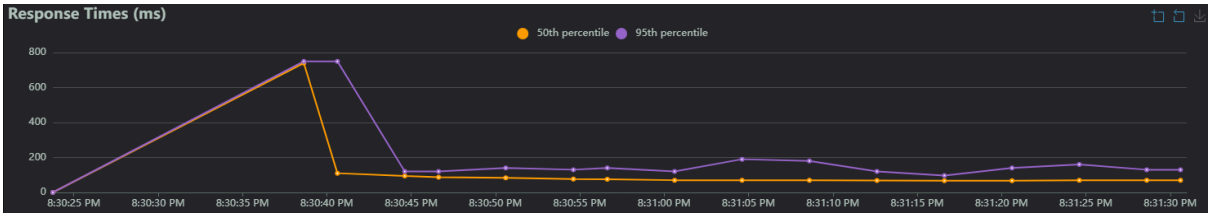
- 2.000 korisnika: Median 21.000ms, 95%ile 33.000ms



Analiza problema i primenjena rešenja:
Glavni uzrok "zagušenja" na ovom endpoint-u nije bila baza podataka, već iscrpljivanje CPU resursa (CPU Exhaustion) zbog istovremenog kriptografskog heširanja, kao i blokiranje niti zbog slanja mejlova i upisa fajlova na disk (I/O Wait).

Kao ključni korak optimizacije, slanje imejla je izbačeno iz glavne izvršne niti (Thread). Takođe, primenjen je niži faktor "cene" za BCrypt heširanje (kao kod logina).

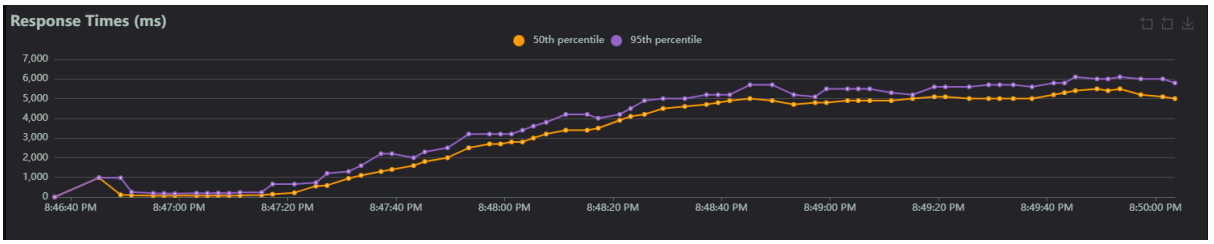
Rezultati nakon optimizacija i uočena ograničenja:
Iako su performanse neznatno poboljšane u nižim kategorijama (na 100 korisnika palo sa 210ms na 70ms), pri testiranju masovnog opterećenja problem je opstao:



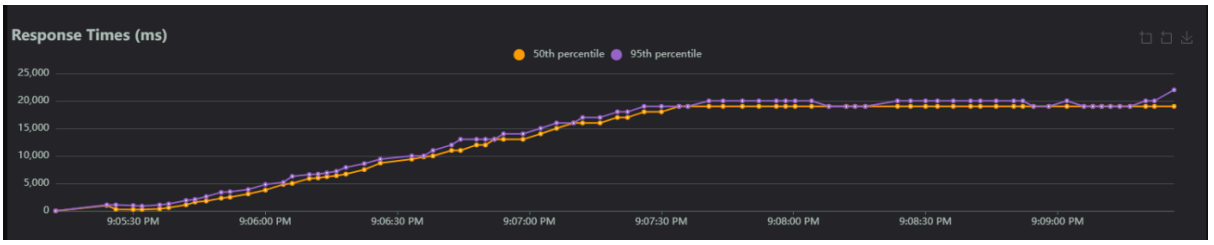
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	2. POST /register	2263	0	70	130	310	83.31	56	750	134	48.7	0
	Aggregated	2263	0	70	130	310	83.31	56	750	134	48.7	0

- 500 korisnika: Median 1.200ms, 95%ile 1.800ms

- 1.000 korisnika: Median 4.500ms, 95%ile 5.600ms

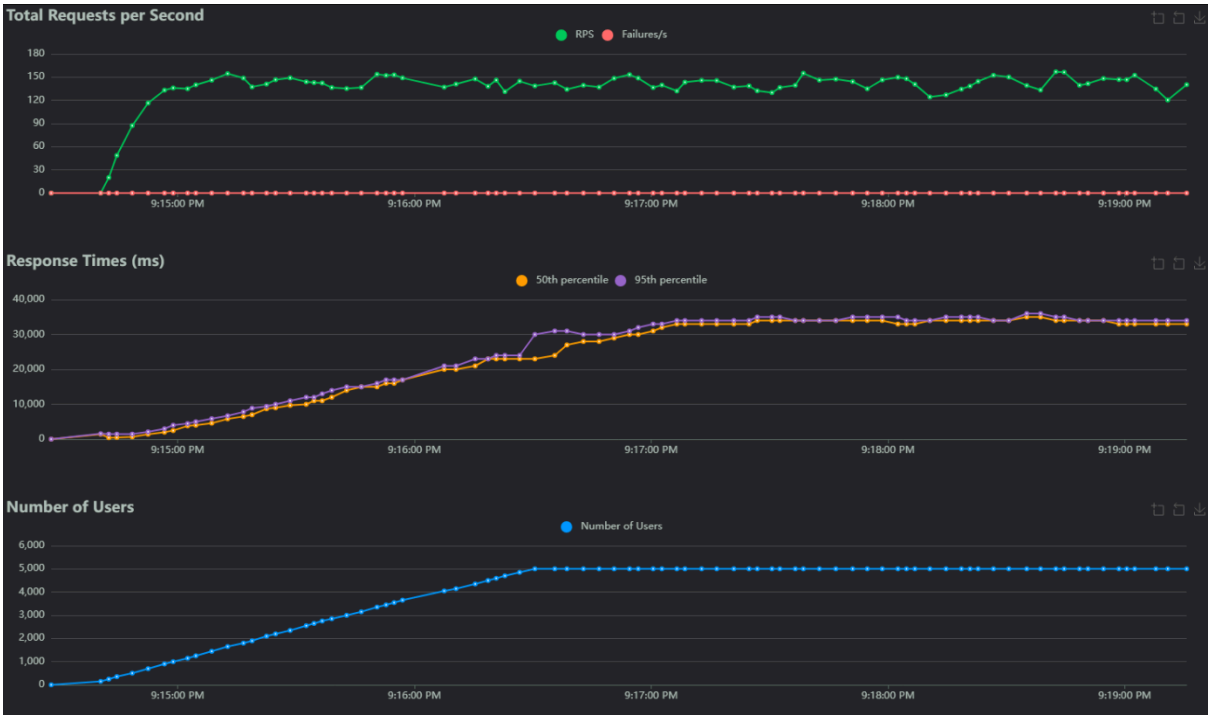


- 2.000 korisnika: Median 11.000ms, 95%ile 13.000ms
- 3.000 korisnika: Median 18.000ms, 95%ile 20.000ms



- 5.000 korisnika: Median 31.000ms, 95%ile 35.000ms

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	2. POST /register	38342	0	31000	35000	35000	24689.22	62	37094	134	140.3	0
	Aggregated	38342	0	31000	35000	35000	24689.22	62	37094	134	140.3	0



Testiranje je namerno zaustavljeno na 5.000 istovremenih korisnika jer je medijana vremena odziva dostigla 33 sekunde. Pretraživači automatski prekidaju vezu usled mrežnog timeout-a kada on pređe granicu od ~30 sekundi. Dalje povećanje korisnika nema praktičnu vrednost za analizu, jer iako aplikacija ne baca direktne greške, sistem postaje neupotrebljiv sa stanovišta korisničkog iskustva. Buduće optimizacije bi morale uključiti prebacivanje snimanja profilnih slika na cloud/fajl server (npr. S3) kako bi se potpuno eliminisao lokalni disk I/O, kao i primenu redova za poruke (RabbitMQ) za odloženu kreaciju korisnika..

2.3. Kreiranje narudžbine: *POST /orders*

Opis i cilj testa:

Cilj ovog testa bio je simulacija situacije iz realnog sveta, gde veliki broj korisnika istovremeno pokušava da rezerviše ili kupi proizvode čija je zaliha ograničena u magacinima. Ovaj endpoint predstavlja jedan od tehnički najkompleksnijih segmenata aplikacije jer zahteva smanjivanje zaliha u magacinu uz strogo poštovanje konzistentnosti i integriteta podataka iz nefunkcionalnog zahteva 6.5.

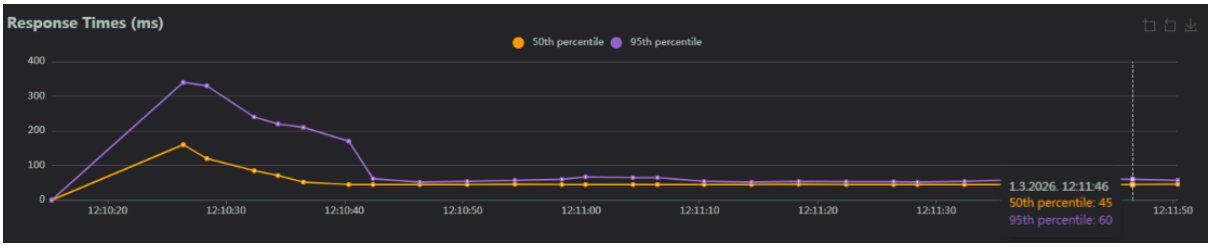
Početne performanse i uočeni problemi:

Prilikom prvog testiranja na osnovnom opterećenju od 100 korisnika, vremena su bila odlična (medijana ~40ms). Međutim, kada je test dignut na 1.000 korisnika, brojka skače stostruko (na 3.100ms), a najvažnije, pojavio se kritičan broj bačenih grešaka od strane sistema.

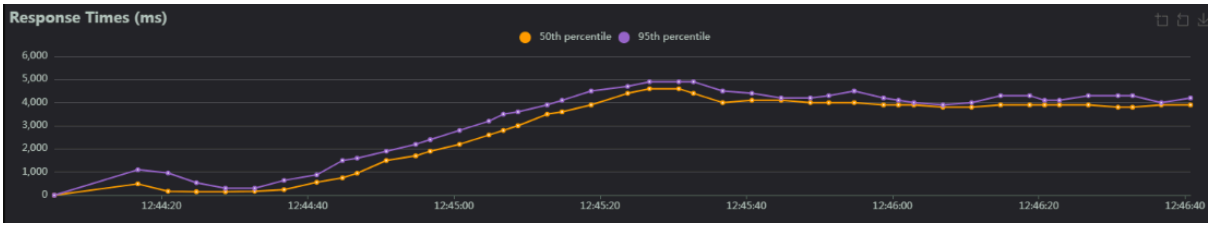
U logovima je uočena Hibernate optimistic locking failure greška (Status 409: CONCURRENT MODIFICATION), što znači da je sistem uspešno uhvatio konflikte transakcija kada više korisnika pokuša istovremeno da naruči isti artikal, odbijajući da upiše netačne podatke.

Vreme odziva

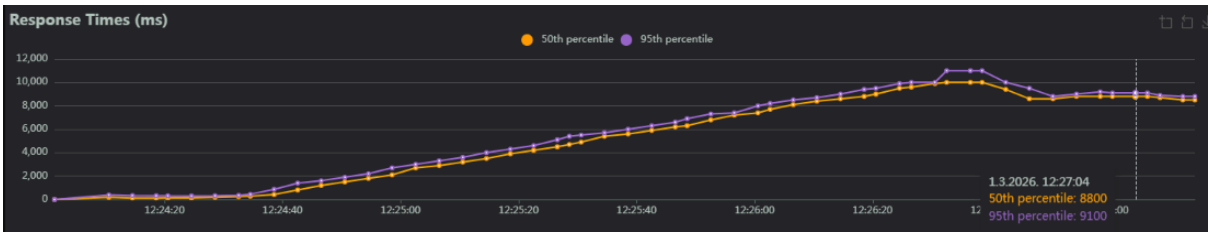
- 100 korisnika: Median 46ms, 95%ile 67ms



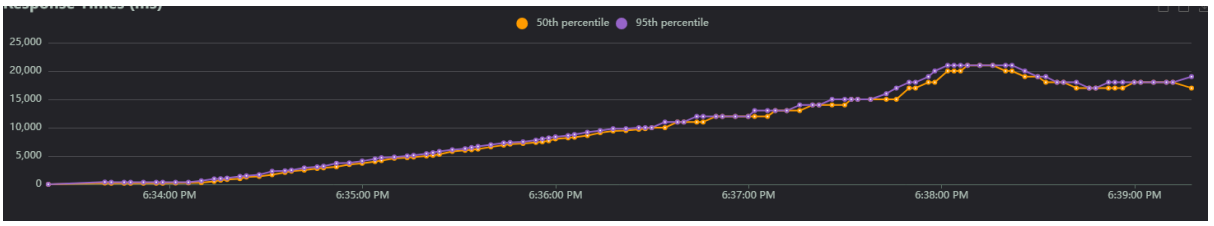
- 1.000 korisnika: Median 3.100ms, 95%ile 4.100ms



- 2.000 korisnika: Median 7.100ms, 95%ile 10.000ms



- 5.000 korisnika: Median 15.000ms, 95%ile 20.000ms



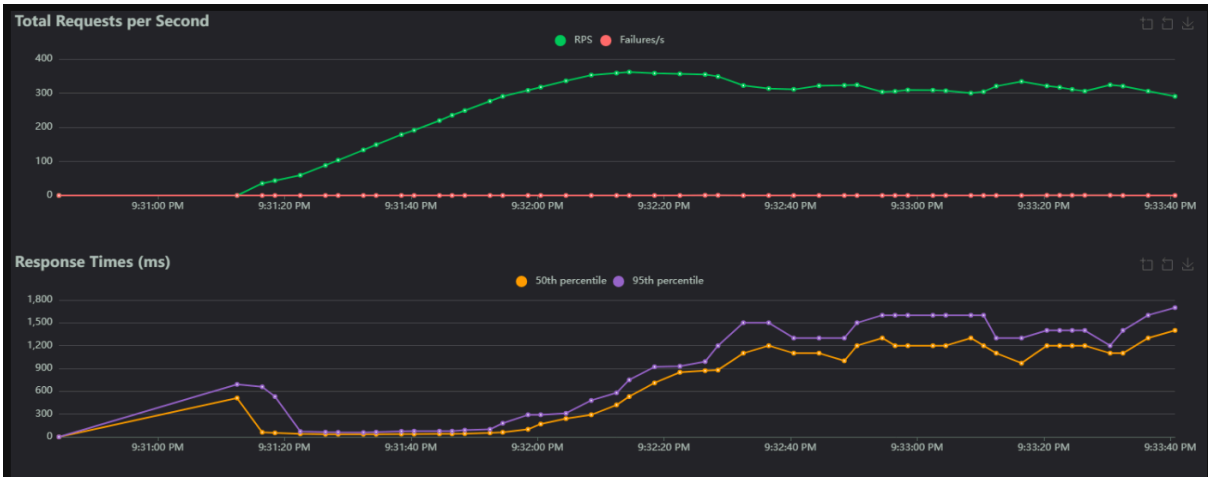
Analize problema i primenjena rešenja:

Pored transakcionih sudara (Serialization failures), usporavanje upita bilo je primarno uzrokovano generisanjem i slanjem fakture putem imejla na samom kraju porudžbine.
Da bi se drastično smanjilo "zaključavanje" baze prilikom ovih konflikata:

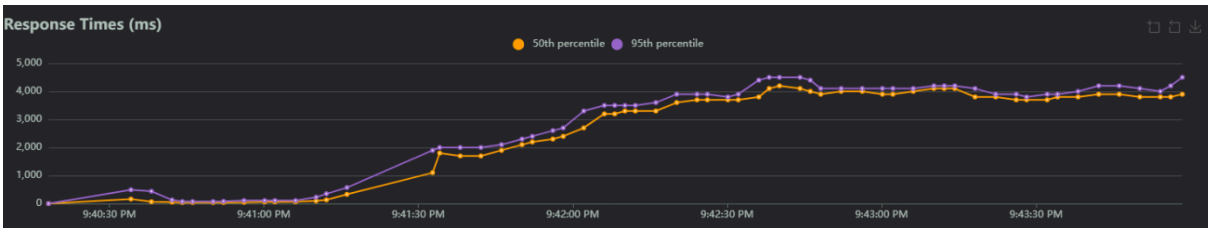
1. Kreiran je Spring Retryable mehanizam: Ukoliko više niti pokuša modifikaciju istih zaliha, umesto automatskog pucanja i prebacivanja greške na korisnika, obezbeđen je automatski ponovni pokušaj izvršenja transakcije (Retry), obezbeđujući glatko procesuiranje bez narušavanja baze.
2. RabbitMQ za asinhronu obradu faktura: Iako je za registraciju samo izvučena asinhrona metoda, za račune je napravljen potpuno nezavisan Topic. Po završetku narudžbine, sistem sada samo emituje događaj na Message Broker-u (RabbitMQ) i vraća odgovor klijentu, dok "Email Listener" asinhrono kreira i šalje PDF fakturu u pozadini.

Rezultati i zaključak:
I pored ovih optimizacija i izbacivanja generisanja računa iz transakcije, zbog izuzetno kompleksne prirode smanjivanja stanja i brojnih validacija, aplikacija i dalje pokazuje veća vremena na masovnim testovima, dok su performanse na manjim skupom za nijansu bolje:

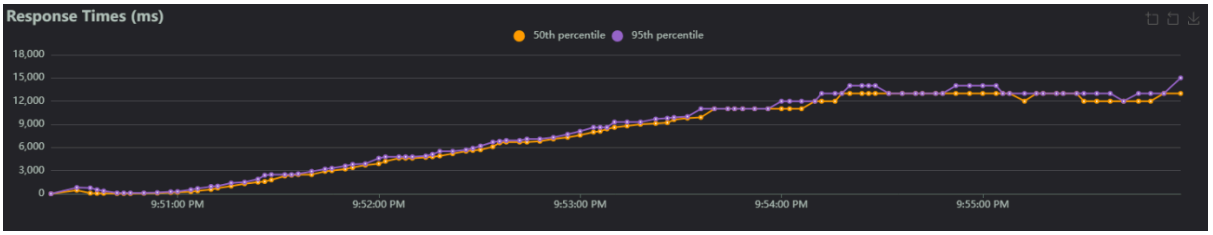
- 1000 korisnika: 950ms, 1500ms



- 2000 korisnika 3600 median, 95% 4200:



- 5000: median 10000 13000



- 7.500 korisnika: Median 12.000ms, 95%ile 21.000ms

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/auth/login	7500	0	8200	18000	19000	8493.29	29	19911	445.11	0	0
POST	3. POST /orders	136484	87	12000	21000	21000	11581.06	26	23107	34.33	303.5	0.5
	Aggregated	143984	87	12000	21000	21000	11420.22	26	23107	55.73	303.5	0.5

Zaključak: S obzirom da na vrhuncu testa sistem drži oko 300 RPS nad strogo kontrolisanim i zaključanim relacijama (Optimistic Lock), trenutna arhitektura je ocenjena kao uspešna. Potvrđeno je da sistem ni u trenucima pucanja konekcija ne gubi narudžbine i ne kreira "fantomsku" zalihu na magacinu.

2.4. Kreiranje magacina: *POST /warehouses*

Opis i cilj testa:

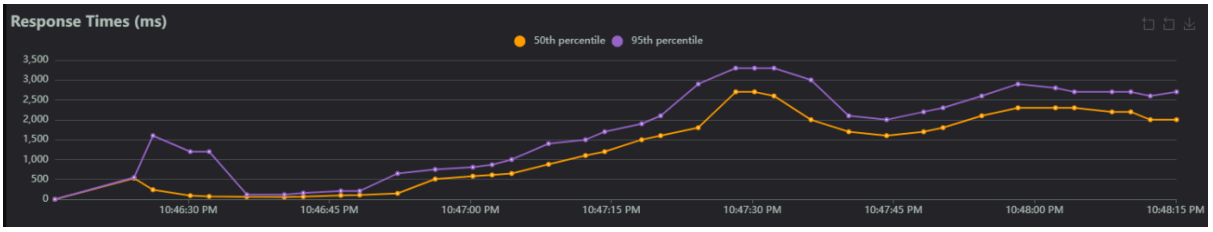
Ovaj endpoint simulira zahtev u kome menadžeri sistema kreiraju nove magacine sa pripadajućim sektorima. Karakteriše ga *Multipart/form-data* tip zahteva, jer se pored složenog JSON objekta (koji uključuje hijerarhijske relacije magacina, grada, države i sektora), istovremeno šalje i jedna ili više slika magacina. Cilj testa bio je ispitati kako se server ponaša pod opterećenjem prilikom obrade *File I/O* operacija u kombinaciji sa relacionalnim upisima.

Početne performanse i uočeni problemi:

Prilikom prvog testa sa 100 istovremenih korisnika, sistem je pokazao vrhunske performanse. Upiti su obrađeni sa medijanom od 51ms. Međutim, ovo vreme je drastično poraslo sa povećanjem broja korisnika, pretežno zbog blokiranja na nivou Connection Pool-a baze podataka.

POST	5. POST /warehouses	1556	0	51	91	230	61.6	33	1314	982.31	37.7	0

Sa 1000 korisnika ~1800ms medijana.



Analizom koda utvrđeno je da se kompletan proces kreiranja (učitavanje grada/države, upload slike na disk, kreiranje entiteta i sektora) odvija unutar jedne @Transactional metode.

Logika za čuvanje slika (fileStorageService.saveWarehouseImage) je izdvojena iz transakcione metode. Transakcija je sada ograničena samo na onaj deo koda koji se direktno upisuje u bazu podataka (nakon što je fajl uspešno sačuvan na disku).

Rezultati na visokom opterećenju (Nakon separacije transakcije):

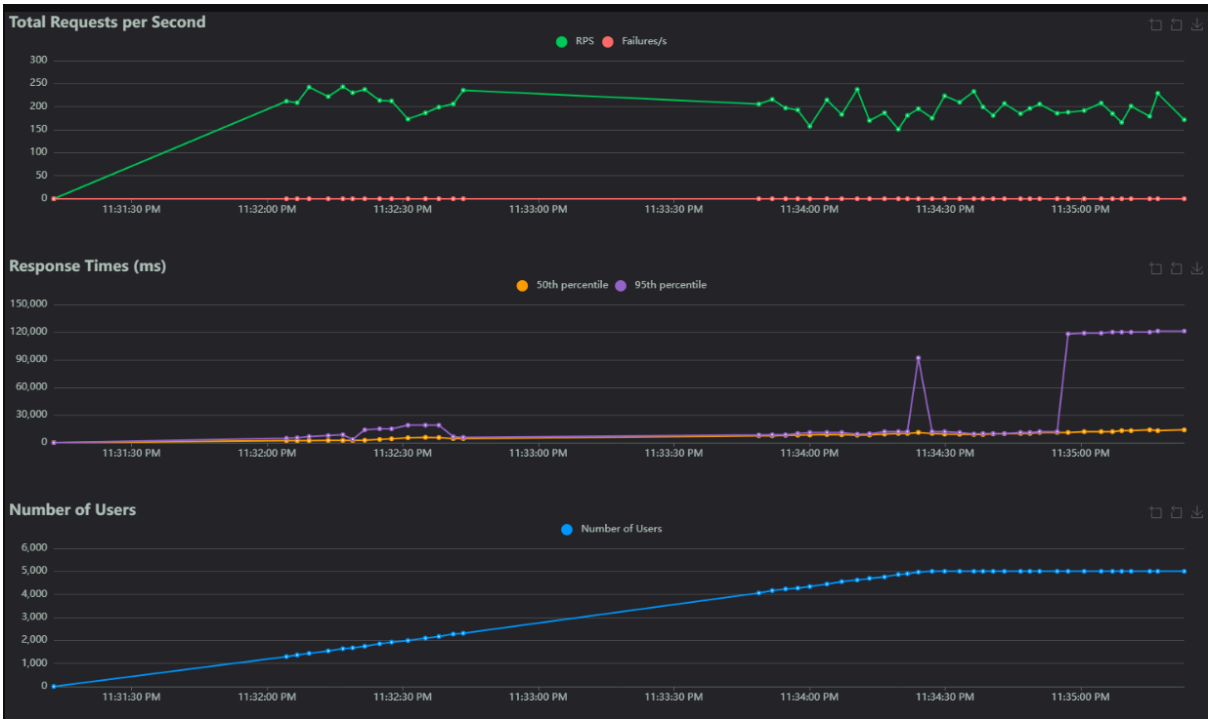
Iako je izdvajanje transakcije sprečilo direktno pucanje baze podataka na nivou konekcija, Locust testovi su pokazali da je usko grlo i dalje prisutno usled samog limita diska da paralelno upisuje veliki broj slika:

- 1.000 korisnika: Median 1.300ms, 95%ile 2.100ms

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/auth/login	1000	0	190	1500	1700	407.21	35	1948	409	0	0
POST	5. POST /warehouses	14899	0	1300	2100	2600	1094.19	38	3497	931.33	223.4	0
	Aggregated	15899	0	1200	2100	2600	1050.98	35	3497	898.48	223.4	0

- 5.000 korisnika: Median 6.400ms, 95%ile 13.000ms

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/api/auth/login	3202	0	29000	121000	122000	45801.1	50	123924	409	15.3	0
POST	5. POST /warehouses	44450	2	6400	13000	14000	6640.26	48	17077	931.29	156.2	0
	Aggregated	47652	2	6500	14000	119000	9271.7	48	123924	896.2	171.5	0



Na opterećenju od 5.000 korisnika, medijana od 6.400ms (6,4 sekunde) je i dalje prihvatljiva u tehničkom smislu s obzirom na to da nema padova (fails) ili grešaka (Connection Refused), ali ukazuje na jasno opterećenje hardverskih resursa na lokalnoj mašini.

Zaključak i predlog budućih unapređenja:

Utvrđeno je da u arhitekturi gde aplikativni server samostalno handluje slike na lokalnom fajl sistemu, nije moguće značajnije unaprediti performanse isključivo kroz prepravku koda na samom bekendu.

U skladu sa najboljim inženjerskim praksama (i zahtevom 6.8 o čuvanju datoteka), jedina prava optimizacija za ovaj problem jeste prebacivanje upload procesa na Cloud infrastrukturu. Ako bi se implementirao S3 bucket, klijent (Frontend aplikacija) bi fajl slao direktno na Cloud servis zaobilazeći naš Backend server, a našem endpoint-u bi prosleđivao samo običan URL string. U tom scenariju, vremena odziva za kreiranje magacina bila bi uporediva sa običnim tekstualnim upitima, bez degradacije performansi sa povećanjem opterećenja.

2.5. Dobavljanje magacina: *GET/warehouses{id}*

Opis i cilj testa:

Ovaj endpoint simulira zahtev u kome menadžeri otvaraju detaljan prikaz specifičnog magacina. Zbog relacione prirode podataka, ORM (*Hibernate*) mora da izvrši JOIN operacije kako bi zajedno sa osnovnim podacima magacina povukao i entitete grada, države, povezane slike, kao i kompletnu listu sektora koji mu pripadaju.

Metodologija testiranja i izazovi:

U prvoj fazi testiranja, simulirani korisnici su pogađali ovaj endpoint, pri čemu je svaki zahtev podrazumevao slanje direktnog SQL upita ka PostgreSQL bazi. Zbog ranijih test slučajeva gde se keširanje pokazalo kao dobra praksa, to sam uradila i za ovaj get zahtev

- 100 korisnika: Median 19ms, 95%ile 37ms

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/auth/login	100	0	29	100	120	35.34	18	124
GET	4. GET /warehouses/{id}	1709	0	19	37	69	26	9	1313

- 1.000 korisnika: Median 18ms, 95%ile 39ms

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/auth/login	1000	0	40	71	100	43.66	20	123
GET	4. GET /warehouses/{id}	31593	0	18	39	62	19.91	7	151

- 5.000 korisnika: Median 1700ms, 95%ile 4400ms

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/auth/login	5000	0	390	8300	12000	2027.61	21	13633
GET	4. GET /warehouses/{id}	271584	0	580	3000	3800	1014.03	7	5761

- 10.000 korisnika: Median 2000ms, 95%ile 5600ms

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/auth/login	8181	0	4500	91000	101000	21879.43	25	105049
GET	4. GET /warehouses/{id}	283496	0	2000	5600	6700	2310.34	7	8783

Analiza i primenjena optimizacija:
Kako su detalji magacina (lokacija, broj sektora) veoma retko podložni izmenama u poređenju sa brojem pregleda, ovaj endpoint je identifikovan kao idealan kandidat za privremeno skladištenje u memoriji.
Da bi se rasteretio disk i procesor baze podataka, implementiran je in-memory keš sistem pomoću Redis-a. Korišćenjem Spring anotacije @Cacheable, aplikacija po prijemu prvog zahteva učitava magacin iz baze i smešta ga u Redis keš. Svi naredni zahtevi ka istom magacinu ne generišu SQL upite, već se željeni podaci serviraju ekstremno brzo, direktno iz radne memorije. (Integritet podataka je osiguran upotrebom @CacheEvict anotacije koja briše keš po izvršenoj izmeni magacina).

Zaključak:
Korišćenjem Redis-a za statičke GET operacije, serveru je omogućeno da pruži stabilan, konstantan i trenutan odgovor klijentima, istovremeno čuvajući dragocene konekcije (*Connection Pool*) ka relacionoj bazi za esencijalne operacije poput kreiranja porudžbina.

2.6. Pretraga i filtriranje magacina *GET/warehouses*

Opis i cilj testa:

Prema zahtevu 5.8 iz specifikacije, menadžerima je omogućeno efikasno pretraživanje liste magacina. Ovaj endpoint prima različite parametre za filtriranje (tekstualna pretraga po nazivu, filtriranje po državi, kao i online/offline status objekta) i vraća rezultate u formatu paginacije.

S obzirom na to da se u bazi potencijalno može nalaziti hiljade magacina, vraćanje svih zapisa odjednom bi dovelo do zagušenja mrežnog saobraćaja i usporavanja frontenda. Zbog toga je cilj ovog testa potvrda stabilnosti paginiranih upita na bazu, kao i demonstracija naprednog keširanja parametrizovanih rezultata pod velikim opterećenjem.

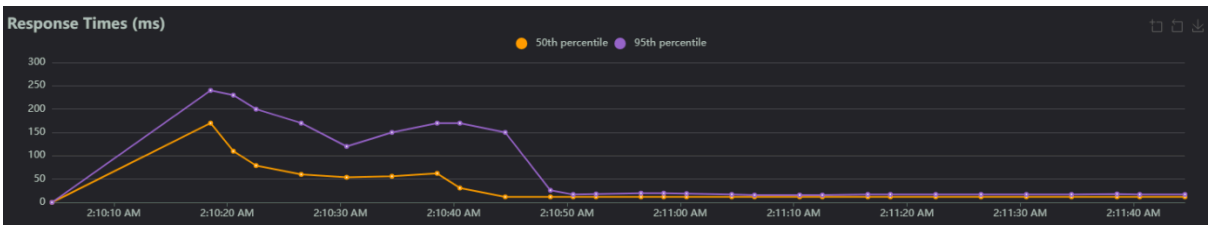
Metodologija i inicijalni pristup (Paginacija):
Kako bi se obezbedila dobra praksa, pretraga je implementirana pomoću ugrađenog Spring Data Pageable interfejsa. Na taj način, SQL upit nad bazom generiše LIMIT i OFFSET klauzule, učitavajući samo onu sekciju podataka (npr. 20 magacina) koja se trenutno prikazuje korisniku. Ovo sprečava iscrpljivanje radne memorije servera (Out of Memory Error) kada bi stotine korisnika istovremeno zahtevalo ispis celokupne liste.
Ipak, i pored paginacije, tekstualna pretraga (LIKE %search%) u PostgreSQL-u je procesorski skupa operacija za bazu, pa se pri većem broju simuliranih korisnika očekivao postepen pad brzine odziva.

Vreme odziva uz primenu paginacije

- 100 korisnika: Median [unesi_rezultat]ms, 95%ile [unesi_rezultat]ms

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/auth/login	100	0	15	32	37	16.81	10	37
GET	7. GET /warehouses	1318	0	12	21	300	16.99	8	497

- 1.000 korisnika: Median 12ms, 95%ile 72ms

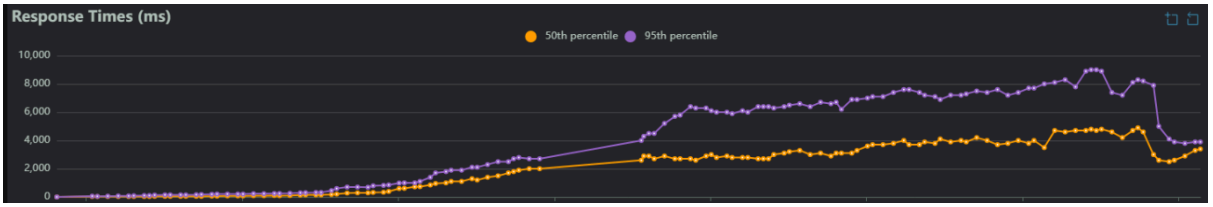


Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/auth/login	1000	0	93	170	200	98.9	27	243
GET	7. GET /warehouses	30357	0	12	72	150	19.79	7	371

- 5.000 korisnika: Median 880]ms, 95%ile 2900ms

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/auth/login	4879	0	400	11000	13000	2506.01	19	14570
GET	7. GET /warehouses	97451	0	720	3000	3600	1028.43	7	4280

- 10.00 korisnika: Median 2500ms, 95%ile 6400ms



Analiza i složena optimizacija keširanjem:
Kako bi se izbeglo višekratno okidanje ovakvih kompleksnih upita u bazi, implementiran je prilagođeni sloj Redis keširanja upotrebom Spring Expression jezika (SpEL). Anotacija @Cacheable generiše kompozitni, jedinstveni ključ (npr. search-country-status-pageNumber-pageSize) u okviru warehouses_filtered keš memorije. Ovim mehanizmom osigurano je da se svaka specifična kombinacija pretrage upita u bazu tačno jednom. Kada stotinu korisnika otvori, na primer, početnu stranicu tabele magacina u Nemačkoj, samo će prvi zahtev dostići bazu podataka, dok će ostalih 99 povući rezultate direktno iz brzog In-Memory Redis keša.

Zaključak:
Uspešno je kombinovano rešenje smanjenja obima prenesenih podataka (Paginacija) i eliminacije redundatnih kalkulacija (Keširanje baze). Sistem sada može besprekorno da odgovori na zahteve velikog broja menadžera koji aktivno pretražuju i filtriraju bazu operativnih magacina, dok se tačnost podataka garantuje pravovremenim "evictovanjem" celokupnog warehouses_filtered direktorijuma svaki put kada se kreira, izmeni ili obriše određeni magacin.

2.7. Kreiranje sektora *POST/warehouses/{warehouseId}/sectors*

Opis i cilj testa:

Cilj testiranja ovog endpoint-a je provera propusne moći servera za standardne INSERT operacije u bazu (bez ometajućih I/O faktora poput učitavanja slika), i ispitivanje do koje

granice bazični sistem može da obrađuje relacione upise pre nego što dođe do čekanja usled zagušenja.

Metodologija testiranja:

Vreme odziva prilikom masovnog upisa sektora

- 100 korisnika: Median 19ms, 95%ile 29ms

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/auth/login	100	0	25	84	95	30.47	14	95
POST	8. POST /warehouses/{id}/sectors	3715	0	19	29	54	21.47	15	373

- 1.000 korisnika: Median 19ms, 95%ile 150ms

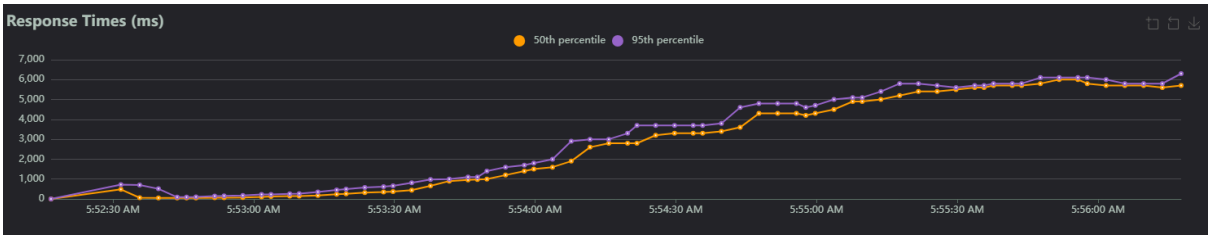
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/auth/login	1000	0	75	140	200	80.06	25	270
POST	8. POST /warehouses/{id}/sectors	24974	0	19	150	240	36.68	12	378

- 3.000 korisnika: Median 2300ms, 95%ile 2900ms

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/auth/login	3000	0	200	2100	2200	638.54	32	2351
POST	8. POST /warehouses/{id}/sectors	60488	0	2300	2900	3000	1814.36	13	3288

- 5.000 korisnika: Median 3700ms, 95%ile 5900

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/auth/login	5000	0	1400	5300	5500	2068.9	20	5651
POST	8. POST /warehouses/{id}/sectors	115826	0	3700	5900	6100	3385	12	6668



- 7.000 korisnika: Median 5000, 95%ile 8400

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/auth/login	7000	0	3200	8400	8500	3508.78	12	8816
POST	8. POST /warehouses/{id}/sectors	132921	0	5000	9600	9700	5041.81	13	9980

- 10.000 korisnika: Median 5000, 95%ile 8400

# Failures	Method	Name	Message
1708	POST	/api/auth/login	ConnectionRefusedError(10061, '[WinError 10061] No connection could be made because the target machine actively refused it.')
7592	POST	8. POST /warehouses/{id}/sectors	ConnectionRefusedError(10061, '[WinError 10061] No connection could be made because the target machine actively refused it.')

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/auth/login	9900	1708	4200	9200	10000	4213.24	18	10156
POST	8. POST /warehouses/{id}/sectors	198429	7592	5900	11000	12000	5779.8	12	12164

Analiza rezultata i tehnička zapažanja:
Pošto ovaj endpoint ne mora da enkodira kriptografske lozinke, niti čeka na spori "Disk I/O" (kako je to bio slučaj kod registracije i kreiranja celog magacina sa slikama), vremena procesiranja na nižim opterećenjima pokazala su izuzetnu brzinu i efikasnost. Vremena čekanja za viša opterećenja nisu idealna, ali su prihvatljiva.

2.8. Pregled detalja sektora *GET/warehouses/{warehouseId}/sectors/{id}*

Opis i cilj testa:

S obzirom na to da obuhvata ukrštanje podataka iz relacione baze (PostgreSQL) sa podacima vezanim za proizvode i agregacijom osnovnog stanja, reč je o tranziciono zahtevnom upitu.

Cilj testa je ispitivanje brzine učitavanja stranice pod pretpostavkom da veliki broj korisnika učitava podatke o specifičnim sektorima istovremeno.

Metodologija testiranja:

Ovde je uvedeno in-memory keširanje korišćenjem kompozitnog ključa (kombinacija warehouseId i sectorId).

Tabela 1: Vreme odziva uz implementiran Redis Cache

- 100 korisnika: Median 11ms, 95%ile 19ms

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/auth/login	100	0	14	70	77	18.93	10	77
GET	5. GET /warehouses/{w_id}/sectors/{s_id}	1663	0	11	19	41	13.06	8	316

- 1.000 korisnika: Median 12ms, 95%ile 100ms

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/auth/login	1000	0	58	110	140	62.8	15	197
GET	5. GET /warehouses/{w_id}/sectors/{s_id}	25278	0	12	100	160	25.52	8	396

- 5.000 korisnika: Median 1000ms, 95%ile 2200ms

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/auth/login	5000	0	320	3300	4000	886.87	16	5226
GET	5. GET /warehouses/{w_id}/sectors/{s_id}	163014	0	1000	2200	2400	1110.88	8	3392

Zaključak:
Slično kao i kod prikaza magacina, Redis in-memory zaštita efikasno "štiti" bazu podataka od nepotrebnih očitavanja. Promena stanja u magacinu (kroz narudžbine ili kreiranje) regularno *evict*-uje ovaj ključ iz memorije kako bi podaci ostali konzistentni.

2.9. Pregled prosečne dostupnosti sektora GET /warehouses/{warehouseId}/analytics/availability

Opis i cilj testa:
Baza mora na osnovu arhiviranih zapisa o statusu da izračuna procentualno i vremensko trajanje onlajn prisustva u određenim vremenskim prozorima.
Zbog prirode status podataka (koji nisu kontinuirana merenja poput temperature, već "State changes"), kalkulacija trajanja predstavlja najsloženiji matematički upit na celoj aplikaciji.

Metodologija testiranja i optimizacija:
Prilikom slanja asinhronih zahteva za dobavljanje parametara dostupnosti (npr. prozor od poslednjih mesec dana, grupisan po nedeljama), aplikacija pristupa InfluxDB-u kako bi povukla boolean vrednosti stanja i njihov *timestamp*, računajući matematičku razliku (delte) između "Online" i "Offline" poruka. Zbog visoke CPU potrošnje pri ovakvim kalkulacijama, ovo je podložno brzom padu performansi prilikom izuzetno visokog "Load"-a ako se ne bi koristio bafer.

Tabela 1: Vreme odziva analitike dostupnosti (InfluxDB State Tracking)

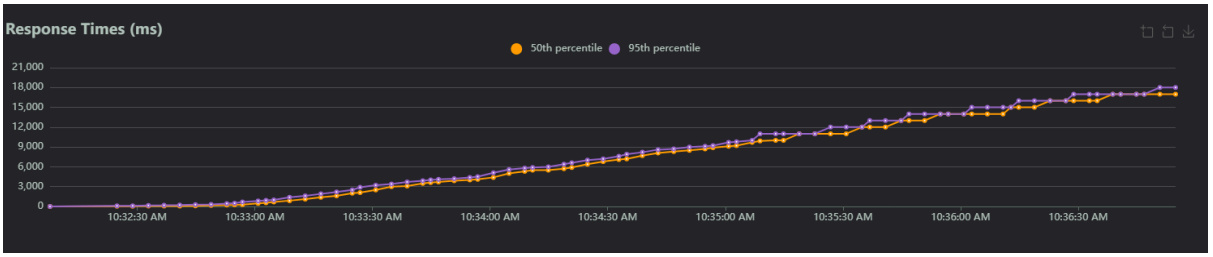
- 100 korisnika: Median 30ms, 95%ile 70ms

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/auth/login	100	0	37	410	420	77.42	22	424
GET	9. GET /analytics/availability	1515	0	30	70	460	44.62	22	1166

- 1.000 korisnika: Median 1100ms, 95%ile 2100ms

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/auth/login	1000	0	77	850	1000	219.36	28	1276
GET	9. GET /analytics/availability	14885	0	1100	2100	2300	1008.88	24	3452

- 5.000 korisnika: Median 8900ms, 95%ile 18000ms



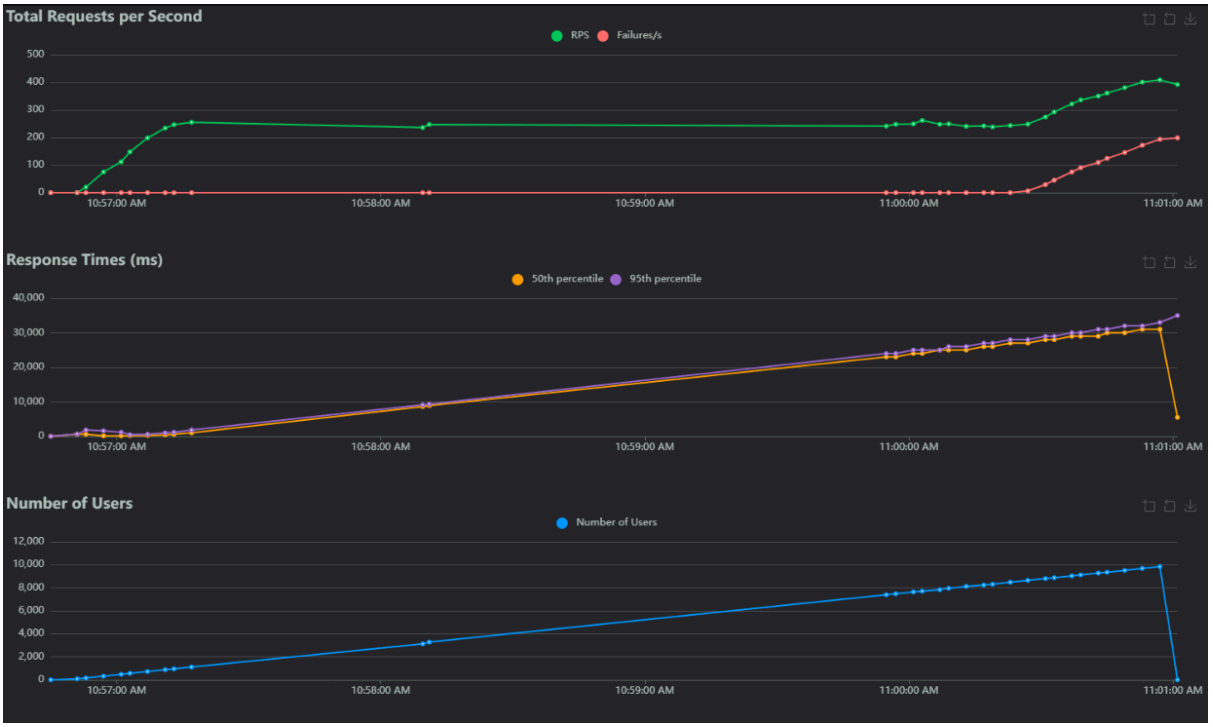
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/auth/login	5000	0	7600	16000	17000	7636.58	14	17543
GET	9. GET /analytics/availability	59885	0	8900	18000	19000	8994.04	23	20317

  - 5.000 korisnika: Napomena: Uvedena je optimizacija kojom se brojanje stanja prebacilo na samu bazu (Flux funkcije group i count), čime je sprečeno OutOfMemory curenje na Spring serveru. Iako ova optimizacija čuva stabilnost sistema, nije drastično spustila medijanu vremena odziva jer dinamičko grupisanje ostaje CPU-intenzivan zadatak za InfluxDB.

GET	9. GET /analytics/availability	50784	0	8500	19000	19000	8951.74	23	20109

- 10.000 korisnika: Median 16000ms, 95%ile 31000ms (dostignuta granica opterećenja)

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/auth/login	9610	1418	11000	29000	30000	12948.01	21	31284
GET	9. GET /analytics/availability	53590	3509	16000	31000	34000	15475.95	25	35243



Zaključak:
Uprkos uspešnoj optimizaciji Flux upita koja je obezbedila potpunu mrežnu i memorijsku stabilnost Java aplikacije, primetno je usporavanje performansi pri ekstremnom saobraćaju (od nekoliko hiljada asinhronih korisnika). Razlog leži u činjenici da je generisanje dinamičkih Time-Series vremenskih agregacija u samoj InfluxDB bazi CPU-intenzivna operacija. Bez obzira na viša vremena odziva pri maksimalnom stresu, arhitektura je proglašena izuzetno stabilnom. Preuzimanjem najtežeg dela posla, baza osigurava da analitika ni u jednom trenutku ne dovede do pucanja niti preopterećenja samog poslovnog (backend) sloja aplikacije, isporučujući uvek tačne i obrađene rezultate.

2.10. Pregled temperatura sektora GET /warehouses/{warehouseId}/analytics/temperature

Opis i cilj testa:
Simulatori generišu ogroman broj zapisa (144 merenja dnevno po sektoru), koji se čuvaju u Time-Series bazi podataka (InfluxDB).
Ovaj endpoint vrši kompleksnu agregaciju (downsampling) miliona zapisa, izračunavajući prosečnu vrednost (MEAN) unutar zadatih vremenskih prozora (po danu, nedelji ili mesecu).
Cilj testa bio je utvrditi brzinu odziva analitike pod paralelnim opterećenjem, kao i sposobnost InfluxDB-a da izdrži agregacione Flux upite.

Metodologija i rezultati testiranja:
Pošto izračunavanje proseka za period od godinu dana zahteva parsiranje preko 50.000 zapisa samo za jedan sektor, podatak je morao biti optimizovan (shodno zahtevu 7.4 koji nalaže primenu downsampling metoda).
Locust je simulirao korisnike koji vrše upite za različite vremenske raspone. InfluxDB (zbog svoje *Time-Series* prirode) optimizovan je za agregacije, pa umesto da se sva sirova

merenja povlače u Spring Boot i tu ručno računaju, Flux upiti su konfigurisani tako da sama InfluxDB baza vrši grupisanje pomoću aggregateWindow(every: 1d, fn: mean). Zbog veličine i specifičnosti upita, rezultati ovih analitičkih HTTP zahteva su takođe predmet Spring keširanja.

Tabela 1: Vreme odziva analitike temperature (Agregacija InfluxDB-a)

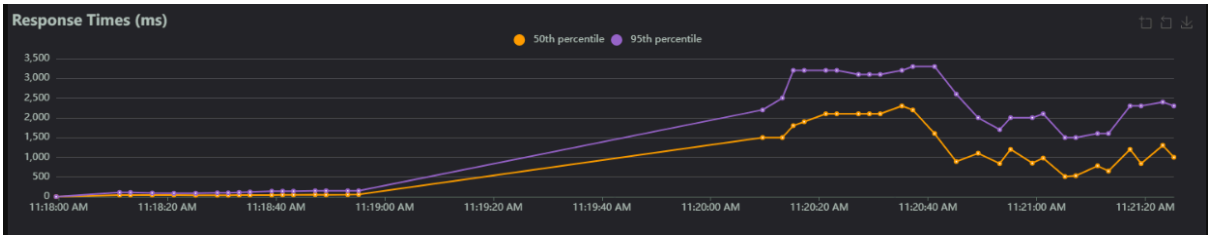
- 100 korisnika: Median 9ms, 95%ile 53ms

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/auth/login	100	0	50	410	430	96.9	24	426
GET	10. GET /analytics/temperature	1108	0	9	53	300	19.94	6	485

- 1.000 korisnika: Median 11ms, 95%ile 87ms

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)
POST	/api/auth/login	1000	0	110	210	270	119.64	26	299
GET	10. GET /analytics/temperature	29600	0	11	87	150	21.48	6	329

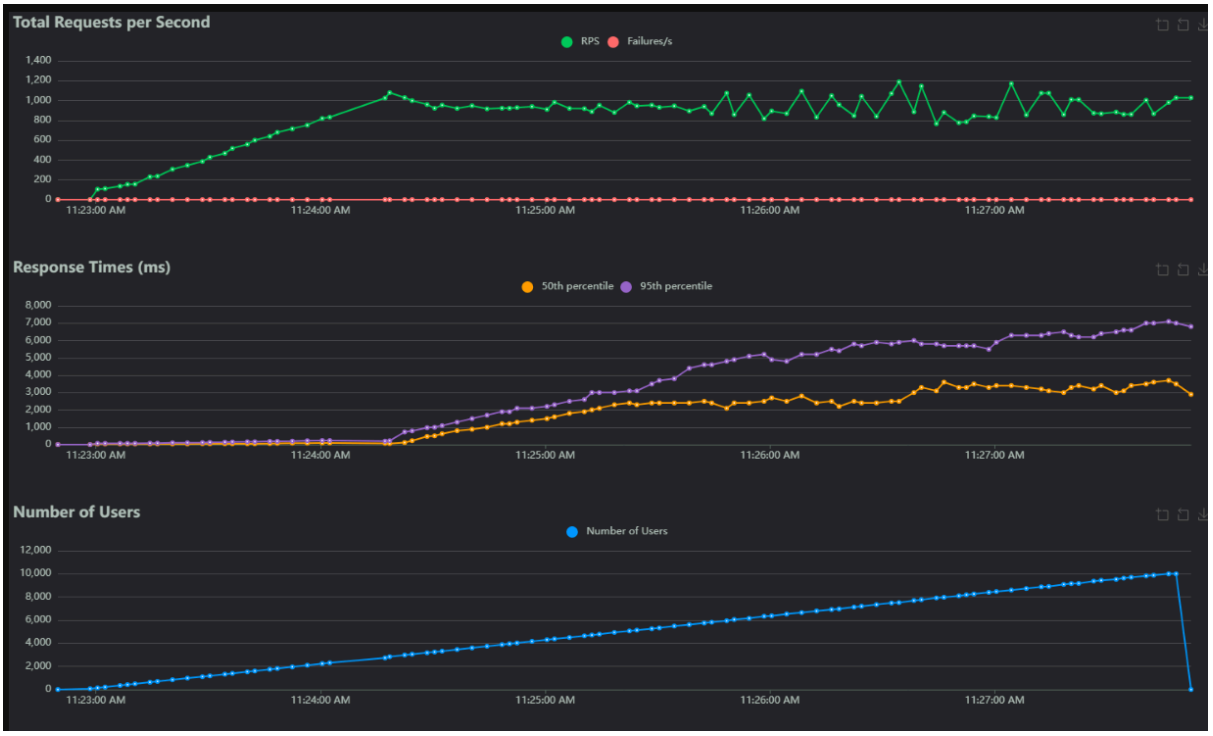
- 5.000 korisnika: Median 700ms, 95%ile 2400ms



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)
POST	/api/auth/login	5000	0	350	9100	11000	2267.17	23
GET	10. GET /analytics/temperature	168113	0	700	2400	3000	891.6	6

- 9.000 korisnika: Median 700ms, 95%ile 2400ms

GET	10. GET /analytics/temperature	238287	0	1600	5400	6500	2005.93	6	8563



Zaključak:
Prebacivanje matematičkog tereta ("downsampling") direktno na Time-Series bazu rezultovalo je odličnim performansama analitike. Grafički podaci se kreiraju efikasno, bez obzira na višemilionsku količinu zapisa u repozitorijumu, te je ispunjen zahtev robusne obrade velikih količina informacija.